GITHUB PROFILE MAKEOVER · ENGINEERING-PRO

Professional GitHub Profile Preview

For **nicksananton0923-create**

Electrical Technology · Renewable Energy · Industrial Maintenance · Arduino & IoT

Files in this preview	1 profile README · 6 repo READMEs · 1 setup guide
Theme palette	Solar Yellow #F5B800 · Circuit Blue #0A4D8C
Rendered	2026 · for review before upload

> This PDF is a rendered preview only. Upload the `.md` files in the `github-profile/` folder to your GitHub repositories to make them live. See *SETUP-GUIDE* at the end of this document for step-by-step instructions.



Building Smart Solutions ⚡ Electrical Technology ✳ Industrial Maintenance ⚙ Arduino & IoT

GitHub nicksananton0923-create Location Sri Lanka Focus Industrial & Embedded
Status Learning & Building

Profile Visitors 0 LinkedIn Connect Email Contact Me

👨‍🔧 About Me

I am an Electrical Technology student in Sri Lanka focused on the practical side of engineering — wiring it up, instrumenting it, measuring it, and writing it down so the next person can repeat the work. My core interests sit at the intersection of three areas: **renewable energy systems (Solar PV)**, **industrial maintenance and equipment monitoring**, and **embedded automation using Arduino and ESP32**. Rather than only studying these topics in theory, I document every project on GitHub so that the documentation becomes part of my professional record.

I work in a deliberately methodical way: I inspect what already exists, I propose changes only after I understand the constraints, and I prefer reusable CSV / Markdown templates over one-off files. The goal is not to ship flashy demos, but to build a body of evidence — a portfolio that an employer, a senior engineer, or a future version of me can read and immediately understand. Every repository on this account follows the same documentation pattern: an accurate README, a clear folder structure, sample anonymized data, and an explicit statement of what has been verified versus what still needs testing.

I am currently growing my skills in **PLC fundamentals, AutoCAD Electrical, MATLAB data analysis, and dashboard design in Excel / Power BI** — see the *Currently Learning* section below for what is on the bench right now.

🛠️ Tech Stack

⚡ Electrical & Energy	**⚙️ Industrial**	**🤖 Embedded / IoT**	**📊 Data & Docs**

Solar PV systems Inverters & cabling Site coordination Electrical safety	HVAC & chillers Preventive maintenance Asset registers Fault logging	Arduino Nano / UNO ESP32 Sensor integration Servo & relay control	Excel dashboards Power BI (learning) Markdown / GitHub CSV templating



🚀 Featured Projects

> These are the repositories I am actively maintaining. Pin 6 of them to your profile (see *Setup Guide*) — the 7th is this profile README itself.

### ⚡ Industrial Air Quality Monitor Arduino Nano-based industrial air-quality monitor with multi-sensor detection, threshold-based alerts, and a compact embedded enclosure.  [→ View Repository] (https://github.com/nicksananton0923-create/industrial-air-quality-monitor)	### 🔥 Smart Fire Detection & Suppression Arduino-based smart fire detection prototype with conceptual automatic suppression — designed for educational demonstrations of fire sensing, emergency alerts, and automated safety control.  [→ View Repository] (https://github.com/nicksananton0923-create/Arduino-smart-fire-detection-suppression)
### 🍅 Tomato Colour Sorting Conveyor Arduino-based smart conveyor belt that detects tomato ripeness using a colour sensor and sorts produce into green, partially-ripe, and red bins via a servo mechanism.  [→ View Repository] (https://github.com/nicksananton0923-create/Arduino-tomato-colour-sorting)	### 🏭 Industrial HVAC Maintenance Professional HVAC documentation framework — asset registers, chiller / AHU / cooling-tower service records, fault logs, and preventive maintenance schedules with reusable CSV templates.  [→ View Repository] (https://github.com/nicksananton0923-create/Industrial-hvac-maintenance)
### 💧 Industrial Water Consumption Monitoring Industrial water consumption analysis project — meter readings, usage calculations, consumption trends, anomaly identification, and reporting dashboards using Excel and Power BI.  [→ View Repository] (https://github.com/nicksananton0923-create/Industrial-water-consumption-monitoring)	### 🔧 Industrial Maintenance Request System Digital maintenance management system for work requests, technician assignments, work order tracking, breakdown records, and maintenance reporting.  [→ View Repository] (https://github.com/nicksananton0923-create/Industrial-maintenance-request-system)

💼 Practical Experience

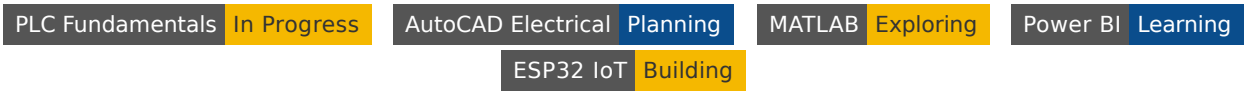
☀️ Solar PV — Hayleys Fentons

Hands-on exposure to solar PV project activities. I supported work around PV modules, inverter installations, DC / AC cabling, site coordination, safety briefings, and material documentation. The role gave me a real-world understanding of how a residential and small-commercial PV system moves from design through commissioning — including the paperwork that has to follow every install to satisfy both the customer and the utility.

🏭 Industrial Maintenance — MAS Vidiyal

I supported general maintenance activities and equipment monitoring at an industrial site. Day-to-day work included maintenance reporting, preventive maintenance follow-through, and structured documentation of equipment conditions and breakdowns. This is where I first saw the gap between *doing* maintenance and *documenting* maintenance — which is exactly what the HVAC and maintenance-request repositories on this profile try to close.

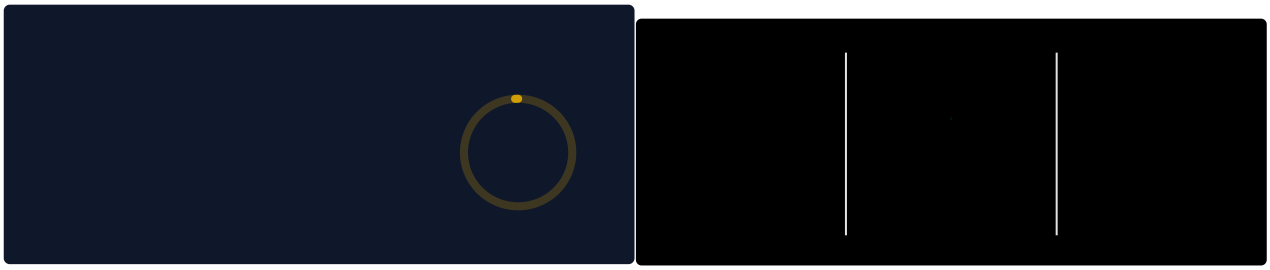
📈 Currently Learning (2026)



2026 Goals:

- [ ] Strengthen electrical fundamentals and troubleshooting skills through hands-on lab work.
- [ ] Complete a production-grade ESP32 monitoring project with cloud telemetry.
- [ ] Build a structured engineering portfolio with clear, repeatable documentation patterns.
- [ ] Develop at least one full automation solution combining sensors, control logic, and a dashboard.
- [ ] Grow as a problem-solving engineer who documents technical work to a professional standard.

📊 GitHub Stats



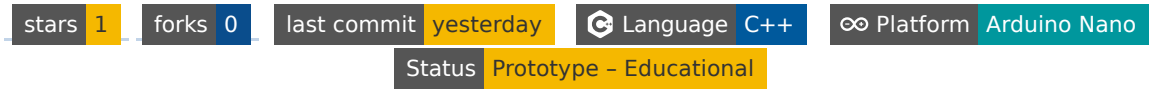


🤝 Connect

GitHub @nicksananton0923-create LinkedIn Connect Email Contact Me Portfolio View

⚡ Learning every day. Building practical skills. Creating solutions with purpose. ⚡

⚡ Industrial Air Quality Monitor **Arduino Nano-based industrial air-quality monitoring and alert system**



📋 Table of Contents

- Overview
- Background
- Key Features
- Hardware Components
- System Architecture
- Repository Structure
- Getting Started
- Usage
- Safety & Compliance Notes
- Roadmap
- License

📝 Overview

The **Industrial Air Quality Monitor** is a compact embedded system built around the Arduino Nano that measures several airborne parameters relevant to industrial environments — including particulate matter, gas concentration, temperature, and humidity — and raises an alert when any reading crosses a pre-set safety threshold. The project was developed as an educational prototype to demonstrate how low-cost open-source hardware can be assembled into a serious monitoring instrument suitable for workshops, factory floors, storage rooms, or any space where airborne contaminants can affect worker safety or product quality.

What makes this repository different from a typical Arduino demo is the documentation discipline behind it. Every sensor in the build has its operating range, calibration expectations, and known limitations written down so that a reader can decide whether the prototype is suitable for their own context. Threshold values are clearly marked as configurable and the alert logic is split out from the sensing logic so that the same firmware can be reused across different sensor combinations without

rewriting the core flow. This makes the project a starting point that can be adapted to gas-specific hazards (CO, CO_2, VOC, dust) rather than a one-off build that only runs on a workbench.

The system is intended as an **educational demonstration**, not a certified industrial safety instrument. It is suitable for student labs, learning projects, and proof-of-concept deployments. Anyone who needs to deploy air-quality monitoring in a regulated industrial environment should use certified, calibrated commercial instruments and follow local occupational health and safety regulations.

🔭 Background

In industrial environments, airborne hazards are among the most difficult risks to manage because they are invisible and accumulate gradually. Workers may be exposed to elevated levels of particulate matter, hazardous gases, or simply to environments where temperature and humidity drift outside a safe band — and there is often no obvious warning until symptoms appear. A small, low-cost monitoring device placed in the right location can provide an early warning that protects both people and equipment, but most commercial options are either too expensive for distributed deployment or too opaque to adapt.

This project was built to address two practical gaps at the same time: first, the need for a low-cost, visible, and audible warning when air quality crosses a threshold; second, the need for a documented, repeatable build that an electrical technology student can read end-to-end and understand. By keeping the firmware modular and the documentation honest about sensor limitations, the repository supports both goals without overpromising what hobby-grade sensors can actually deliver.

✨ Key Features

- **Multi-sensor air-quality monitoring** — integrates gas, particulate, temperature, and humidity sensors on a single Arduino Nano, with each sensor polled on its own recommended interval. The design intentionally separates the *sensing* layer from the *alerting* layer so that swapping sensors only requires updating the relevant module, not rewriting the main loop.
- **Threshold-based alert system** — every measured parameter has a configurable safe range defined in a single configuration section at the top of the firmware. When any reading falls outside its safe range, the system triggers a visible (LED) and audible (buzzer) alarm. Thresholds can be tuned for different environments without modifying the alert logic itself.
- **Real-time serial output** — all sensor readings are written to the serial monitor in a simple key:value format, making it easy to log readings, plot trends, or pipe the data into a PC-side dashboard for further analysis. The format is intentionally human-readable so that troubleshooting does not require a custom tool.
- **Modular firmware structure** — the codebase is split into clearly named modules (sensor drivers, threshold configuration, alert manager, main loop) rather than a single monolithic `.ino`

file. This keeps the firmware readable and makes it straightforward to extend the system with new sensors or different output channels (LCD, Wi-Fi, SD card logging).

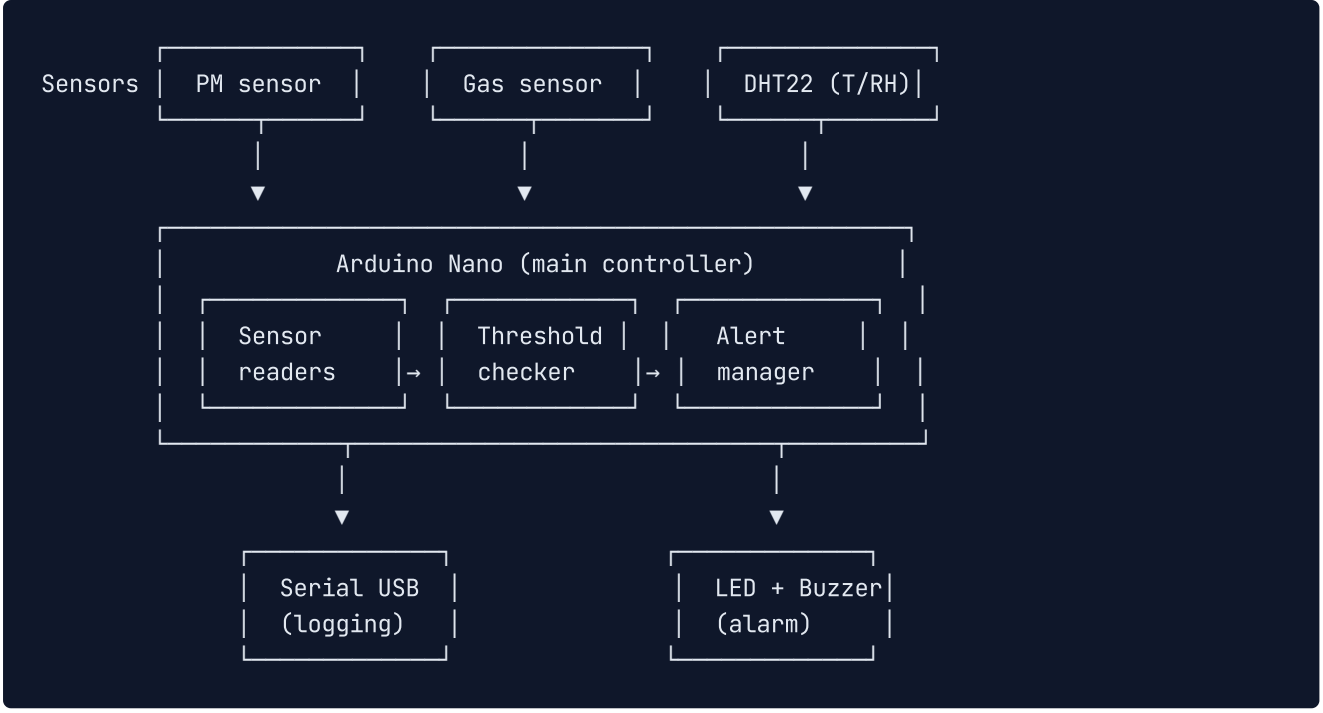
- **Compact, low-power build** — designed around the Arduino Nano so the full system fits in a small enclosure and can be powered from a standard 5 V USB source or a 7–12 V DC adapter. This makes it practical to deploy in multiple rooms without significant wiring work.

🔌 Hardware Components

Module	Component	Role	Notes
Controller	Arduino Nano (ATmega328P)	Main MCU, runs firmware	5 V logic, USB-powered
Particulate	SDS011 or similar PM sensor	PM2.5 / PM10 measurement	UART interface
Gas	MQ-series sensor (e.g. MQ-135)	Broad gas concentration reading	Analog output, requires calibration
Climate	DHT22 (or DHT11)	Temperature + humidity	Single-wire digital interface
Alert — visual	LED (red + green)	Status indicator	Green = OK, Red = alarm
Alert — audible	Piezo buzzer	Audible alarm on threshold breach	Active or passive, configurable
Power	5 V USB or 7–12 V DC jack	System power	USB is sufficient for development

> Component choices are intentionally flexible. The firmware is structured so that an alternate sensor with a similar interface can be substituted without rewriting the main loop.

🧱 System Architecture



The architecture follows a **sense** → **check** → **alert** pipeline. Each sensor module produces a reading on its own cadence, hands it to the threshold checker, which compares it against the configured safe range, and the alert manager decides whether to raise, hold, or clear the alarm. Serial output runs in parallel so the user always sees the live readings regardless of alarm state. This separation is the key to the firmware's reusability: a different sensor only changes the first block, a different alarm mechanism only changes the last block.

📂 Repository Structure

```
industrial-air-quality-monitor/
├── README.md                  # This file
├── /firmware                  # Arduino .ino source
│   ├── air_quality_monitor.ino
│   └── /modules                # Sensor drivers, threshold, alert
├── /docs
│   ├── wiring-guide.md         # Pin-by-pin wiring reference
│   ├── sensor-notes.md        # Calibration + limitations per sensor
│   └── threshold-config.md    # How to tune thresholds
├── /data
│   └── sample-reading-log.csv  # Sample anonymized serial output
└── LICENSE
```

> If your current repository does not yet have all these folders, use this structure as the target layout. Add folders as you build out the documentation — do not delete existing files.

🚀 Getting Started

Prerequisites

- Arduino IDE 1.8.13 or newer (or Arduino CLI)
- Arduino Nano board (or compatible clone with the same MCU)
- USB cable for programming and serial monitoring
- Sensors listed in *Hardware Components* above

Installation

1. **Clone the repository**
   `bash git clone https://github.com/nicksananton0923-create/industrial-air-quality-monitor.git cd industrial-air-quality-monitor`

2. **Open the firmware in Arduino IDE**
   - Open `firmware/air_quality_monitor.ino` in the Arduino IDE.
   - From *Tools* → *Board*, select **Arduino Nano**.
   - From *Tools* → *Processor*, select the matching variant (e.g. ATmega328P).
   - From *Tools* → *Port*, select the USB port the Nano is connected to.

3. **Install required libraries**
   - Use *Sketch* → *Include Library* → *Manage Libraries* and install any libraries referenced in the firmware (e.g. the DHT sensor library by Adafruit).

4. **Wire the sensors** following the wiring guide in `docs/wiring-guide.md`.

5. **Upload the firmware** and open the Serial Monitor at the baud rate declared at the top of the sketch (default 9600).

🎯 Usage

Once the firmware is uploaded and the sensors are wired, the monitor runs autonomously. The default operating loop is:

1. Each sensor is polled on its recommended interval (e.g. DHT22 every 2 seconds, PM sensor every 1 second).
2. Readings are printed to the serial monitor in `key:value` format.
3. Each reading is compared to its configured threshold.
4. If any reading breaches its threshold, the LED turns red and the buzzer pulses.
5. When all readings return to safe range, the alarm clears and the LED returns to green.

To customize thresholds, edit the configuration block at the top of `air_quality_monitor.ino`:

```
// ═══ Threshold configuration ═══
const float PM25_MAX   = 35.0;   // µg/m³
const float PM10_MAX   = 75.0;   // µg/m³
const float GAS_MAX    = 400.0;  // ppm (sensor-specific)
const float TEMP_MIN   = 15.0;   // °C
const float TEMP_MAX   = 35.0;   // °C
const float HUM_MIN    = 30.0;   // %
const float HUM_MAX    = 70.0;   // %
```

⚠️ Safety & Compliance Notes

- **Educational use only.** This device is not a certified industrial safety instrument. Do not use it as the sole means of hazard detection in a regulated environment.
- **Sensor calibration.** MQ-series gas sensors require warm-up time and periodic calibration. Readings from these sensors are indicative, not laboratory-grade. Always cross-check against a calibrated reference instrument before relying on the numbers.
- **Threshold tuning.** Default thresholds are illustrative. Real industrial deployments should follow the workplace exposure limits published by the relevant occupational health authority in your jurisdiction.
- **Power safety.** Use a properly rated power supply. For continuous deployment, prefer a regulated 5 V DC source over USB power from a PC.

🗺 Roadmap

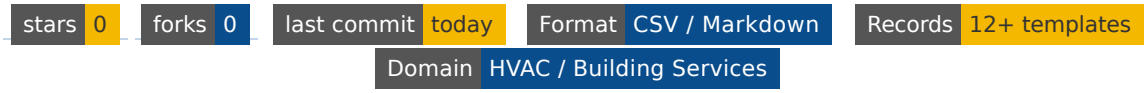
- [ ] Add a small OLED display for local readout of current values.
- [ ] Add an SD card module for offline logging when no PC is connected.
- [ ] Add an ESP32 variant with Wi-Fi telemetry to a simple dashboard.
- [ ] Add a calibration helper sketch to simplify MQ sensor baseline tuning.
- [ ] Add a chart-friendly CSV export format for downstream Excel / Power BI analysis.

📄 License

This project is released under the MIT License unless a separate `LICENSE` file in the repository specifies otherwise. See the repository root for the actual license text.

⚡ Built for learning. Documented for the next engineer. ⚡ Project by [@nicksananton0923-create]
(https://github.com/nicksananton0923-create) · Electrical Technology · Renewable Energy · Automation

🏭 Industrial HVAC Maintenance **Professional HVAC, Chiller, AHU & Cooling System maintenance documentation framework**



📋 Table of Contents

- Overview
- Background & Motivation
- What's Inside
- Repository Structure
- Templates (CSV)
- Documentation (Markdown)
- Diagrams (PNG)
- Getting Started
- How to Use the Templates
- Safety & Accuracy
- Roadmap
- License

📝 Overview

The **Industrial HVAC Maintenance** repository is a complete documentation framework for organizing and managing maintenance work on industrial heating, ventilation, air-conditioning, and cooling systems. It contains reusable CSV templates for every common record a building-services team needs — asset registers, chiller service records, AHU inspection checklists, cooling-tower logs, fault and corrective action logs, preventive maintenance schedules, and more — alongside Markdown documentation that explains the purpose of each field, the methodology behind each inspection, and how the templates fit into a coherent maintenance program.

This is not a software project in the traditional sense; it is a structured set of files that an HVAC maintenance team, a building services manager, or a student of facility engineering can drop into a shared folder and start using immediately. The templates are deliberately vendor-neutral and use plain column names so they open cleanly in Excel, Google Sheets, LibreOffice Calc, or any CSV-aware tool. Every record includes clear field descriptions, sensible defaults, and explicit notes on

which fields are mandatory versus optional, which makes them easy to onboard for new technicians without long training sessions.

The repository is being built incrementally — every file that exists is already production-useful, and every file that is still on the roadmap is explicitly called out in the *Roadmap* section. Nothing in this repository should be considered final until it has been reviewed by a qualified maintenance engineer for the specific equipment and jurisdiction in which it will be used.

🔭 Background & Motivation

Industrial HVAC systems — chillers, air handling units, cooling towers, pumps, fans, and the control networks that tie them together — represent some of the most expensive and most failure-prone equipment in any modern building. A single chiller failure can take an entire process plant offline; a single poorly documented AHU inspection can mean a filter is replaced a month late, an indoor-air-quality complaint is dismissed, or a bearing fails without warning. The cost of these events is almost always higher than the cost of the documentation that would have prevented them.

And yet, in many facilities, maintenance records are scattered: some live on paper, some live in spreadsheets, some live in the heads of senior technicians, and some are not recorded at all. The result is that when a technician leaves, knowledge walks out the door; when an audit arrives, no one can prove what was inspected and when; and when a fault recurs, no one can trace whether the same fault has been seen before. This repository was built to provide a simple, reusable starting point for fixing that gap. It does not replace a full CMMS, but it gives a team a structured, vendor-neutral place to start collecting records in a consistent format — which is the first step toward any future digital system.

📦 What's Inside

This repository currently includes:

- **12 CSV templates** covering the full maintenance record lifecycle from asset registration through inspection, service, fault logging, and preventive maintenance scheduling.
- **6 Markdown documentation files** covering system overviews, inspection methodology, preventive maintenance planning, recordkeeping practice, control logic and wiring, and safety and compliance.
- **5 PNG diagrams** including the refrigerant cycle, chilled-water system, BMS network architecture, control wiring block, and a Gantt-style preventive maintenance schedule.
- **1 anonymized sample records file** so readers can see what a completed dataset looks like without exposing any real facility data.
- **Standard project files** — `CHANGELOG.md`, `CONTRIBUTING.md`, `LICENSE`, `.gitignore` — to make the repository straightforward to contribute to and fork.

📂 Repository Structure

```
Industrial-hvac-maintenance/
├── README.md                          # This file
├── CHANGELOG.md                       # Version history
├── CONTRIBUTING.md                    # How to contribute
├── LICENSE                            # License file
├── .gitignore
│
├── # ═══ CSV templates ═══
├── hvac-asset-register.csv
├── ahu-inspection-checklist.csv
├── chiller-service-record.csv
├── cooling-tower-log.csv
├── filter-replacement-log.csv
├── refrigerant-service-log.csv
├── pump-fan-bearing-inspection.csv
├── operating-readings.csv
├── fault-corrective-log.csv
├── preventive-maintenance-schedule.csv
├── sample-records.csv                 # Anonymized sample data
│
├── # ═══ Markdown documentation ═══
├── system-overview.md
├── inspection-methodology.md
├── preventive-maintenance-planning.md
├── recordkeeping-guide.md
├── control-logic-and-wiring.md
├── safety-and-compliance.md
│
└── # ═══ Diagrams (PNG) ═══
    ├── refrigerant-cycle.png
    ├── chilled-water-system.png
    ├── bms-network-architecture.png
    ├── control-wiring-block.png
    └── pm-schedule-gantt.png
```

📊 Templates (CSV)

Template	Purpose
`hvac-asset-register.csv`	Master list of every HVAC asset: ID, location, type, model, serial, install date, OEM, service contact.
`ahu-inspection-checklist.csv`	Air handling unit inspection: filter, coil, belt, fan, drain pan, dampers.

`chiller-service-record.csv`	Chiller service log: refrigerant, oil, electrical, controls, performance.
`cooling-tower-log.csv`	Cooling tower inspection: water treatment, fill, drift eliminator, fan.
`filter-replacement-log.csv`	Filter cleaning / replacement history: asset, type, pressure drop, date, technician.
`refrigerant-service-log.csv`	Refrigerant handling: type, quantity added / removed, leak check, technician certification.
`pump-fan-bearing-inspection.csv`	Bearing inspection: vibration, temperature, lubrication, sound.
`operating-readings.csv`	Routine operating readings: temperature, pressure, humidity, flow, current.
`fault-corrective-log.csv`	Fault and corrective action log: symptom, cause, action, parts, downtime.
`preventive-maintenance-schedule.csv`	PM schedule: asset, task, frequency, last done, next due, responsible.
`sample-records.csv`	Anonymized sample rows showing how the templates are used in practice.

> Each CSV uses simple, plain-English column headers and includes a header row that any spreadsheet tool can read directly. Field meanings and recommended conventions are documented in `recordkeeping-guide.md`.

📄 Documentation (Markdown)

Document	Purpose
`system-overview.md`	High-level overview of an industrial HVAC system and its components.
`inspection-methodology.md`	How to inspect each component type — what to look at, in what order, and what to record.
`preventive-maintenance-planning.md`	How to build a PM schedule from the templates, including frequency selection and prioritization.
`recordkeeping-guide.md`	The reference for every column in every CSV — mandatory vs. optional, units, and example values.

`control-logic-and-wiring.md`	Notes on BMS / control logic and wiring conventions, including the diagrams below.
`safety-and-compliance.md`	Safety reminders, refrigerant handling rules, and confidentiality guidance.

🖼 Diagrams (PNG)

Diagram	What it shows
`refrigerant-cycle.png`	Standard vapor-compression refrigerant cycle with labeled components.
`chilled-water-system.png`	Chilled water loop: chiller, pumps, AHU coils, expansion tank.
`bms-network-architecture.png`	BMS / BACnet network architecture with controllers and field devices.
`control-wiring-block.png`	Block diagram of typical HVAC control wiring conventions.
`pm-schedule-gantt.png`	Gantt-style representation of a typical annual PM schedule.

🚀 Getting Started

As a maintenance team

1. **Clone or download** the repository:
   `bash git clone https://github.com/nicksananton0923-create/Industrial-hvac-maintenance.git`
2. **Pick the templates you need** — most teams start with `hvac-asset-register.csv`, `ahu-inspection-checklist.csv`, and `fault-corrective-log.csv`.
3. **Open the CSVs in Excel, Google Sheets, or LibreOffice Calc** — the headers are plain English and the templates are immediately usable.
4. **Read** `recordkeeping-guide.md` to understand every column before you start filling the templates with real data.
5. **Replace sample data carefully** — never commit confidential facility readings to a public fork. Use anonymized copies for any shared version.

As a student

1. Read `system-overview.md` first for the big picture.

2. Read `inspection-methodology.md` to understand how each component is actually inspected.
3. Open the CSVs to see how records are structured.
4. Read `recordkeeping-guide.md` to understand why each field exists.
5. Try filling in `sample-records.csv` with hypothetical data to test your understanding.

🎯 How to Use the Templates

- **One row per record.** Each CSV is structured so one row represents one event (one inspection, one service, one fault). This keeps the data tidy and easy to filter or chart.
- **Asset IDs are sacred.** Use the same asset ID across every CSV so records can be joined. The asset register is the source of truth for what each ID means.
- **Units must be explicit.** Every numeric field has a documented unit in `recordkeeping-guide.md`. Never write a number without a unit, and never mix units in the same column.
- **Dates are ISO 8601.** Use `YYYY-MM-DD` for all dates so the CSVs sort and filter correctly across locales.
- **Technician and signature.** Every record has a `technician` column and (where applicable) a `reviewed_by` column. Accountability is the whole point.

⚠️ Safety & Accuracy

- **Refrigerant handling must be done by certified personnel.** The `refrigerant-service-log.csv` template is for record-keeping only — filling it in does not authorize anyone to handle refrigerants.
- **HVAC electrical work is hazardous.** Any template that includes electrical readings assumes the work is being done by a qualified electrician or technician.
- **OEM manuals override these templates.** Equipment-specific limits, service intervals, and procedures from the OEM manual always take precedence. These templates are vendor-neutral; they cannot capture every model-specific requirement.
- **Confidentiality.** Never commit real facility readings, customer names, or proprietary information to a public repository. Use the `sample-records.csv` pattern as a guide for anonymizing your own data.

🗺 Roadmap

- [ ] Add `boiler-inspection-checklist.csv` for heating-side coverage.
- [ ] Add `iaq-monitoring-log.csv` for indoor air-quality readings.

- [ ] Add a dashboard build guide (Excel / Power BI) that visualizes the CSVs.
- [ ] Add a Python utility script to validate CSVs against the schema in `recordkeeping-guide.md`.
- [ ] Translate key documentation into Tamil for bilingual use.

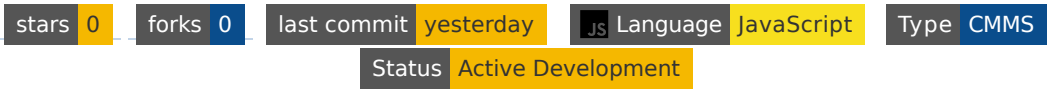
📄 License

This project is released under the MIT License unless the `LICENSE` file in the repository root specifies otherwise. The templates and documentation are free to use, modify, and distribute — including for commercial facility maintenance work — provided the license terms are respected.

🏭 Document it once. Find it forever. 🏭 Project by [@nicksananton0923-create]

(https://github.com/nicksananton0923-create) · Electrical Technology · Renewable Energy · Automation

🔧 Industrial Maintenance Request System **Digital maintenance management — work requests, technician assignments, work-order tracking & reporting**



📋 Table of Contents

- Overview
- Background
- Key Features
- Architecture
- Repository Structure
- Data Model
- Getting Started
- Usage
- Configuration
- Security & Privacy
- Roadmap
- License

📝 Overview

The **Industrial Maintenance Request System** is a lightweight Computerized Maintenance Management System (CMMS) designed for small and medium industrial facilities that need a structured way to handle maintenance work requests without investing in a heavy enterprise platform. The system covers the full lifecycle of a maintenance task: a worker submits a request, a supervisor reviews and approves it, a technician is assigned, the work is carried out and documented, the work order is closed, and the resulting record is archived for future reporting and audit. Every step is logged so that the team can later answer questions such as "how long did this fault take to close?", "which technician closed the most work orders last month?", or "which asset is generating the most repeat requests?".

The project is built in JavaScript because the front-end needs to be approachable for any team member to use on a phone, tablet, or laptop, and because the underlying data model is simple enough that a heavier stack would add more complexity than value. The interface focuses on clarity over feature density: a maintenance technician opening the system on a phone in a noisy plant room

should be able to see exactly what is assigned to them today and update a work order status with two taps, not navigate a multi-level menu.

This repository is being built incrementally, with the data model and core workflow documented first and the user interface layers added on top. Every planned feature is listed in the *Roadmap* so that readers can see what already works and what is still in progress. Nothing in the system should be considered production-ready until it has been reviewed for the specific facility where it will be deployed, including any required access controls, backups, and integration with existing site systems.

🔭 Background

In most small industrial facilities, maintenance requests flow through some combination of paper forms, phone calls, WhatsApp messages, and whiteboards. Each of those channels has the same fundamental problem: there is no structured record. A technician who receives a request by phone has no written record of what was requested, by whom, when, with what priority, or against which asset. When the same fault recurs a month later, no one can easily discover that it has been seen before. When a new technician joins the team, they have no documented history to learn from. When the facility manager is asked "how many breakdowns did we have last quarter?", the answer is usually a guess.

A lightweight CMMS solves this by providing a single place where every maintenance request becomes a structured record. The request captures who asked, what they asked for, which asset is affected, how urgent it is, and when it was reported. The record then flows through a defined lifecycle — approved, assigned, in progress, completed, closed — with each transition logged with a timestamp and the responsible person. This gives the team an audit trail, a learning base, and a reporting surface that none of the ad-hoc channels can provide. The goal of this repository is to make that capability accessible to a small facility team without requiring them to evaluate, purchase, and integrate a commercial CMMS.

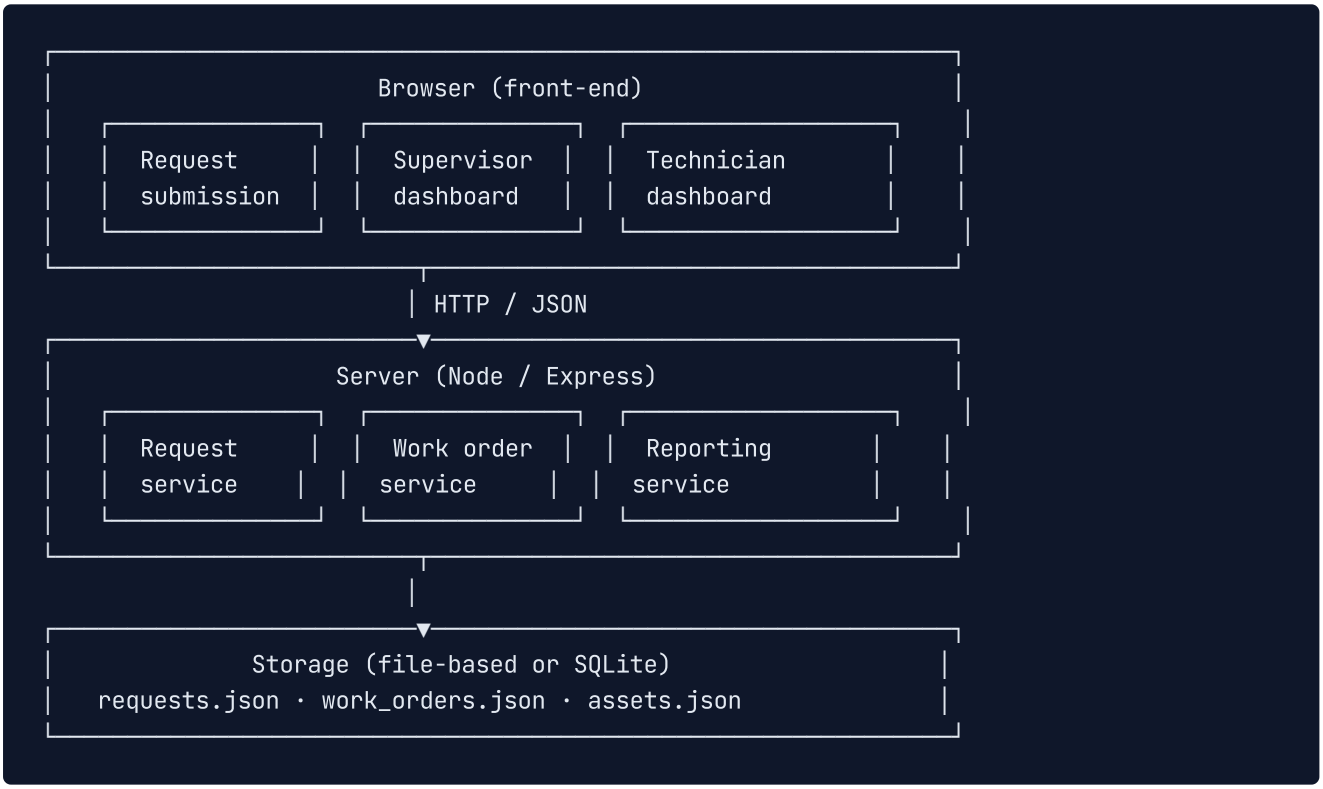
✨ Key Features

- **Work request submission** — any worker can submit a maintenance request from any device. The form captures the requester, the affected asset, the problem description, the urgency, and an optional photo. The form is intentionally short so that submission takes less than a minute.
- **Approval workflow** — a supervisor reviews new requests and either approves, rejects, or requests more information. Approved requests are converted to work orders and made available for assignment. The approval step creates a paper trail and prevents the system from being flooded with low-value or duplicate requests.
- **Technician assignment** — a supervisor assigns each work order to a technician. The technician sees their assigned work orders on their own dashboard, sorted by priority and due date.
- **Work order tracking** — the technician updates the work order status as they progress: accepted, in progress, parts ordered, completed, cannot complete. Each status change is

timestamped and logged. The technician can also record parts used, time spent, and a completion note.

- **Breakdown records** — when a work order is a breakdown (unplanned failure) rather than a planned maintenance task, the system captures additional fields: downtime start, downtime end, root cause, corrective action, and parts replaced. This data feeds the breakdown analysis reports.
- **Reporting** — built-in reports show open work orders by status, work orders closed per technician in a given period, top recurring assets, and total downtime by asset. Reports are exportable to CSV for further analysis.
- **Asset register integration** — work orders are linked to a simple asset register so that historical work orders for any asset can be retrieved in one click. The asset register is the same one used by the HVAC and water-monitoring repositories, so all three systems share a common asset vocabulary.

🧱 Architecture



The system is intentionally storage-light: it can run on file-based JSON storage for a small team or be upgraded to SQLite when concurrency or reporting volume grows. This keeps deployment trivial — the system can be hosted on a small local server, a Raspberry Pi, or any low-cost container host.

📂 Repository Structure

```
Industrial-maintenance-request-system/
├── README.md
├── package.json
├── server.js                    # Node / Express entry point
├── /src
│   ├── /routes
│   │   ├── requests.js          # Submit + list endpoints
│   │   ├── work-orders.js       # Assignment + status update
│   │   └── reports.js           # Reporting endpoints
│   ├── /services
│   │   ├── request-service.js
│   │   ├── work-order-service.js
│   │   └── report-service.js
│   └── /storage
│       ├── requests.json        # Local dev storage
│       ├── work-orders.json
│       └── assets.json
├── /public                      # Front-end (HTML / CSS / JS)
│   ├── index.html               # Request submission form
│   ├── supervisor.html          # Supervisor dashboard
│   └── technician.html          # Technician dashboard
├── /docs
│   ├── data-model.md            # Work request + work order schema
│   ├── workflow.md              # Status lifecycle
│   └── deployment.md            # How to deploy
└── LICENSE
```

🗃 Data Model

The system has three core entities:

Entity	Purpose	Key fields
`Request`	A submitted maintenance request	request_id, requester, asset_id, problem, urgency, status, created_at
`WorkOrder`	An approved request being worked on	work_order_id, request_id, technician, status, parts, time_spent, completed_at
`Asset`	An item of equipment	asset_id, name, location, type, install_date

Requests become work orders on approval. Work orders are linked to assets so historical work orders can be retrieved per asset.

🚀 Getting Started

Prerequisites

- Node.js 18 or newer
- npm or yarn

Installation

1. **Clone the repository**
   `bash git clone https://github.com/nicksananton0923-create/Industrial-maintenance-request-system.git cd Industrial-maintenance-request-system`
2. **Install dependencies**
   `bash npm install`
3. **Start the server**
   `bash npm start`
4. **Open the app** in your browser at `http://localhost:3000` (or the port shown in the console).
5. **Open the request form** at `/index.html`, the supervisor dashboard at `/supervisor.html`, and the technician dashboard at `/technician.html`.

🎯 Usage

Requester

Open the home page, fill in the request form (asset, problem, urgency, optional photo), and submit. You will receive a request ID that can be used to track status later.

Supervisor

Open the supervisor dashboard to see new requests awaiting approval. Approve or reject each request. Approved requests become work orders that can be assigned to a technician. Use the reports view to see open work orders, completed work orders, and recurring assets.

Technician

Open the technician dashboard to see work orders assigned to you, sorted by priority and due date. Update the status as you progress: accepted → in progress → completed. Record parts used, time spent, and a completion note before closing.

⚙️ Configuration

Configuration is stored in `config.json` at the repository root:

```
{
  "port": 3000,
  "storage": "file",
  "dataDir": "./src/storage",
  "assetRegisterPath": "./src/storage/assets.json",
  "defaultUrgencyLevels": ["low", "medium", "high", "critical"],
  "defaultStatusFlow": ["submitted", "approved", "assigned", "in_progress", "completed", "c
}
```

The `storage` field can be set to `file` (default) or `sqlite` to switch storage backends without changing application code.

🔒 Security & Privacy

- **Authentication.** This prototype currently has no built-in authentication. Before deploying in a real facility, add at least a basic role-based login (requester / supervisor / technician / admin).
- **Backups.** If you use file-based storage, back up the JSON files daily. If you use SQLite, back up the database file daily.
- **Confidentiality.** Do not store personal data (full names, phone numbers) unless your facility's data protection policy permits it. Prefer employee IDs over names.
- **Audit logging.** Every status change should be logged with the actor and timestamp. The data model includes this; make sure your deployment preserves it.

🗺 Roadmap

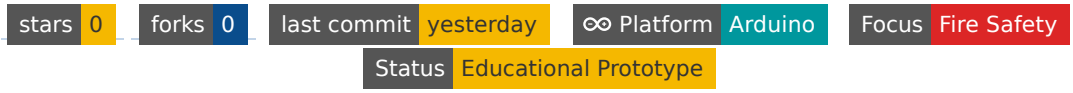
- [ ] Add authentication + role-based access control.
- [ ] Add a SQLite storage backend.
- [ ] Add CSV export for all reports.
- [ ] Add email / SMS notifications on assignment.
- [ ] Add a mobile-friendly PWA wrapper.
- [ ] Add a public read-only "request status" page for requesters.
- [ ] Translate the user interface into Tamil for bilingual use.

📄 License

This project is released under the MIT License unless the `LICENSE` file in the repository specifies otherwise.

🔧 Request it. Approve it. Assign it. Close it. Report on it. 🔧 Project by [@nicksananton0923-create]
(https://github.com/nicksananton0923-create) · Electrical Technology · Renewable Energy · Automation

🔥 Smart Fire Detection & Suppression System **Arduino-based smart fire detection prototype with conceptual automatic suppression**



📋 Table of Contents

- Overview
- Background
- Key Features
- Hardware Components
- System Architecture
- Repository Structure
- Getting Started
- Usage
- Safety & Disclaimer
- Roadmap
- License

📝 Overview

The **Smart Fire Detection & Suppression System** is an Arduino-based prototype designed to demonstrate how embedded electronics can be used to detect the early signs of a fire and trigger an automated safety response. The project combines multiple flame and smoke sensors with control logic that decides when to raise an alarm and when — conceptually — to activate a suppression mechanism such as a water spray, a CO_2 release, or an extinguisher trigger. The system is built as an **educational demonstration** of fire sensing, emergency alerting, and automated safety control; it is not a commercial fire suppression product.

The repository documents the prototype end-to-end: the sensor choices and their limitations, the control logic that fuses multiple sensor inputs to reduce false alarms, the alarm path (visual + audible + remote alert), and the conceptual suppression path that explains how the same logic would interface with a relay-actuated extinguisher in a real deployment. The threshold logic is intentionally separated from the actuation logic so that the same firmware can be reused with different sensor combinations or different suppression endpoints without rewriting the decision core.

For students and educators, the project provides a clean, well-documented example of how to design a safety-critical embedded system the right way: every assumption is written down, every

threshold is configurable, and every limitation is explicitly called out so that the reader can make an informed decision about adapting the work to their own context.

🔭 Background

Fire is one of the most time-critical hazards in any industrial or domestic environment. The difference between a near-miss and a serious incident often comes down to seconds — how quickly the fire is detected, how reliably the alarm is raised, and whether the suppression response starts before the fire has had time to spread. Traditional smoke detectors are excellent for homes but limited in industrial environments where dust, fumes, or heat variations can trigger false alarms. A multi-sensor approach that combines flame detection, smoke detection, and temperature sensing produces a more reliable decision than any single sensor on its own.

This project was built to demonstrate that multi-sensor approach in a way that is reproducible by an electrical technology student. The prototype uses a flame sensor for direct flame detection, a smoke / gas sensor for early combustion products, and a temperature sensor for ambient heat rise. The firmware fuses these three signals using a simple voting system — at least two of the three sensors must agree that a fire condition exists before the alarm escalates — which dramatically reduces false positives from a single noisy sensor while still responding quickly to a real fire event.

✨ Key Features

- **Multi-sensor fire detection** — combines a flame sensor (IR-based), an MQ smoke / gas sensor, and a temperature / humidity sensor. Each sensor contributes one vote to the decision core, so a single noisy reading does not trigger a false alarm. This voting approach mirrors the redundancy used in professional fire detection panels.
- **Threshold-based decision core** — every sensor has a configurable threshold defined at the top of the firmware. The decision logic compares each reading to its threshold, counts how many sensors are in alarm state, and escalates only when the configured minimum number of sensors agree (default: 2 of 3). This makes the system robust against false positives from a single sensor.
- **Three-stage alarm** — when a fire condition is detected, the system raises a visual alarm (red LED + status indicator), an audible alarm (piezo buzzer), and a remote alert output (relay or digital output) that can be wired to a wider alarm system. The remote output is what would interface with a suppression actuator in a real deployment.
- **Conceptual suppression path** — the firmware includes a commented-out suppression module that demonstrates how a relay-actuated extinguisher or water spray would be triggered after a configurable delay. The module is intentionally left inactive in the educational build so that no high-current actuator is energized during classroom demonstrations.
- **State-machine firmware structure** — the main loop is structured as a small explicit state machine (`IDLE → DETECT → ALARM → (optional) SUPPRESS → COOLDOWN → IDLE`) rather

than a tangled `if/else` chain. This makes the control flow easy to read in a classroom setting and easy to extend with new states.

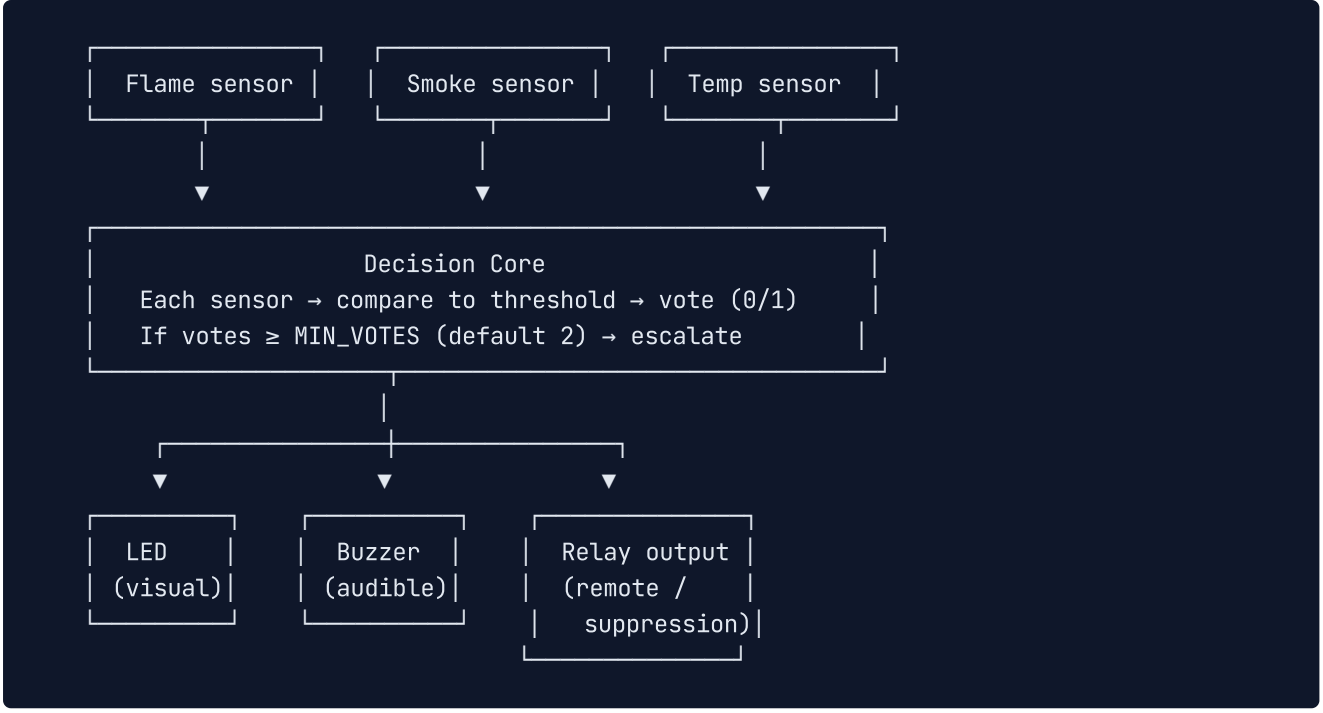
- **Full documentation** — every sensor, every threshold, and every assumption is written down. The documentation explicitly states what has been tested versus what remains conceptual so that anyone adapting the project knows exactly where the boundaries of the prototype are.

🔌 Hardware Components

Module	Component	Role	Notes
Controller	Arduino UNO or Nano	Main MCU	Either works; UNO is easier for classroom demos
Flame sensor	KY-026 flame sensor (IR)	Direct flame detection	Digital + analog output
Smoke sensor	MQ-2 smoke / gas sensor	Combustion gas detection	Analog output, requires warm-up
Climate sensor	DHT22 or LM35	Temperature rise detection	DHT22 also adds humidity
Alert — visual	Red LED + green LED	Status	Green = OK, Red = alarm
Alert — audible	Piezo buzzer	Audible alarm	Active or passive
Remote alert	Relay module (5 V)	Dry contact for external alarm / extinguisher	**De-energized by default**
Suppression (conceptual)	Relay + solenoid valve / CO_2 trigger	Water spray or gas release	**Disabled in educational build**

> The suppression path is intentionally left as a commented-out module. Do **not** connect any high-current or hazardous actuator without supervision and a proper enclosure.

🧱 System Architecture



The decision core implements a simple two-of-three voting rule. A fire event is only declared when at least two sensors report a threshold breach within the same polling window. This reduces false positives from a single noisy sensor (e.g. smoke from cooking triggering the MQ-2 alone) while still catching a real fire event that affects multiple sensors simultaneously.

📂 Repository Structure

```
Arduino-smart-fire-detection-suppression/
├── README.md                  # This file
├── /firmware
│   ├── fire_detection.ino     # Main sketch
│   └── /modules
│       ├── sensors.cpp        # Sensor reader functions
│       ├── decision.cpp       # Voting / threshold logic
│       ├── alarm.cpp          # LED + buzzer control
│       └── suppression.cpp    # Conceptual actuator path (disabled)
├── /docs
│   ├── wiring-guide.md
│   ├── sensor-placement.md
│   └── suppression-safety.md  # Why suppression is disabled by default
├── /data
│   └── sample-test-runs.csv   # Sample test scenarios + outcomes
└── LICENSE
```

🚀 Getting Started

Prerequisites

- Arduino IDE 1.8.13 or newer
- Arduino UNO or Nano board
- USB cable
- Sensors listed in *Hardware Components*

Installation

1. **Clone the repository**
   `bash git clone https://github.com/nicksananton0923-create/Arduino-smart-fire-detection-suppression.git cd Arduino-smart-fire-detection-suppression`

2. **Open the firmware**
   - Open `firmware/fire_detection.ino` in the Arduino IDE.
   - Select the correct board and port from the *Tools* menu.

3. **Install required libraries**
   - Use *Sketch* → *Include Library* → *Manage Libraries* and install any libraries referenced in the firmware.

4. **Wire the sensors** following `docs/wiring-guide.md`.

5. **Upload the firmware** and open the Serial Monitor at the baud rate declared at the top of the sketch.

🎯 Usage

Educational demonstration mode (default)

In the default build, the suppression module is **disabled**. When a fire condition is detected:

1. The red LED lights up and the green LED turns off.
2. The piezo buzzer pulses.
3. The relay output energizes — wire this to a small external lamp or visible indicator for the demo.
4. A status message is written to the Serial Monitor explaining which sensors triggered the alarm.

Configurable thresholds

Edit the configuration block at the top of `fire_detection.ino`:

```
// ═══ Sensor thresholds ═══
const int  FLAME_THRESHOLD     = 600;   // Analog reading (sensor-specific)
const int  SMOKE_THRESHOLD     = 400;   // Analog reading (ppm estimate)
const float TEMP_THRESHOLD     = 50.0;  // °C

// ═══ Decision core ═══
const int  MIN_VOTES_TO_ALARM  = 2;     // out of 3 sensors
const unsigned long ALARM_HOLD_MS = 5000; // hold alarm after sensors clear

// ═══ Suppression (DISABLED by default) ═══
const bool SUPPRESSION_ENABLED = false; // set true ONLY in supervised deployment
```

Test scenarios

Use a lighter, a candle, or a heat gun at a safe distance to trigger each sensor individually and verify the voting logic. See `docs/sensor-placement.md` for safe distances and recommended test scenarios.

⚠️ Safety & Disclaimer

- **Educational prototype only.** This system is **not** a certified fire detection or suppression product and must not be used as the sole means of fire protection in any environment.
- **Suppression is disabled by default.** Enabling the suppression path energizes a relay that can switch high-current or hazardous loads. Only enable it in a supervised deployment with a proper enclosure, fusing, and risk assessment.
- **Sensor limitations.** Hobby-grade flame and smoke sensors have known false-positive and false-negative patterns. Do not use the prototype as a substitute for certified smoke detectors in residential or commercial premises.
- **Test safely.** Always test the sensors in a controlled environment with a fire extinguisher on hand. Never test the suppression path with a real hazard.
- **Compliance.** Any real deployment must follow local fire safety regulations and use certified components. This project does not constitute engineering advice.

🗺 Roadmap

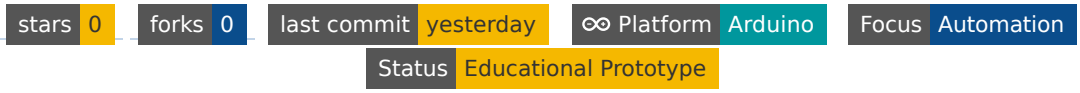
- [ ] Add a small OLED display for local status readout.
- [ ] Add an ESP32 variant with Wi-Fi alerting (SMS / Telegram / email).
- [ ] Add a log-to-SD-card module for post-event analysis.
- [ ] Add a calibration helper sketch to baseline each sensor.
- [ ] Add a documented, supervised suppression test plan.

📄 License

This project is released under the MIT License unless a separate `LICENSE` file in the repository specifies otherwise.

🔥 Detect early. Alert reliably. Suppress safely. 🔥 Project by [@nicksananton0923-create](https://github.com/nicksananton0923-create) · Electrical Technology · Renewable Energy · Automation

🍅 Tomato Colour Sorting Conveyor **Arduino-based smart conveyor belt for tomato ripeness detection and automatic sorting**



📋 Table of Contents

- Overview
- Background
- Key Features
- Hardware Components
- System Architecture
- Repository Structure
- Getting Started
- Usage
- Calibration & Tuning
- Safety & Limitations
- Roadmap
- License

📝 Overview

The **Tomato Colour Sorting Conveyor** is an Arduino-based automation project that detects the ripeness of a tomato using a colour sensor, classifies it into one of three categories — **green**, **partially ripe**, or **red** — and uses a servo-driven diverter to push each tomato into the correct bin as it travels along a small conveyor belt. The project demonstrates how affordable embedded hardware can be assembled into a complete sensing-and-actuation pipeline suitable for small-scale agricultural grading, classroom demonstrations of industrial automation, and proof-of-concept workflows for produce handling.

The repository documents the full build, including the colour sensor selection and calibration approach, the conveyor mechanism, the servo-actuated sorting gate, and the firmware that ties everything together. The classification logic is intentionally simple and explainable: the firmware reads red / green / blue component values from the colour sensor, computes a single ripeness score, and classifies against two configurable thresholds. This keeps the logic transparent so a

reader can audit it end-to-end, while still being flexible enough that the same firmware can be retuned for other produce (oranges, bananas, capsicum) by changing the calibration constants.

The project is built as an educational prototype and is **not** intended for production-scale commercial packing. It is suitable for student lab work, science fairs, and concept demonstrations in agricultural automation courses. As with the other repositories on this account, every assumption and every limitation is written down so that the next reader can adapt the work safely and confidently.

🔭 Background

Manual sorting of produce by ripeness is slow, inconsistent, and labor-intensive. In a small-scale operation — a farm gate, a cooperative packing shed, or a teaching kitchen — there is rarely a budget for an industrial vision system, but there is a real need for a low-cost, repeatable way to separate ripe from unripe produce so that each batch goes to the right destination (immediate sale, ripening storage, or processing). Colour is one of the simplest and most reliable signals for tomato ripeness, which makes a colour-sensor-based sorter an ideal first automation project for an agricultural technology course.

The build sits at a useful intersection of three learning areas: **embedded sensing** (calibrating a colour sensor against real-world produce), **mechanical actuation** (timing a servo gate so the right tomato lands in the right bin), and **control logic** (deciding when to read the sensor, when to actuate the gate, and how to handle the belt's continuous motion). Each of those three layers is documented separately in this repository so the project can be used as a teaching tool for any one of those topics in isolation, not only as a complete system.

✨ Key Features

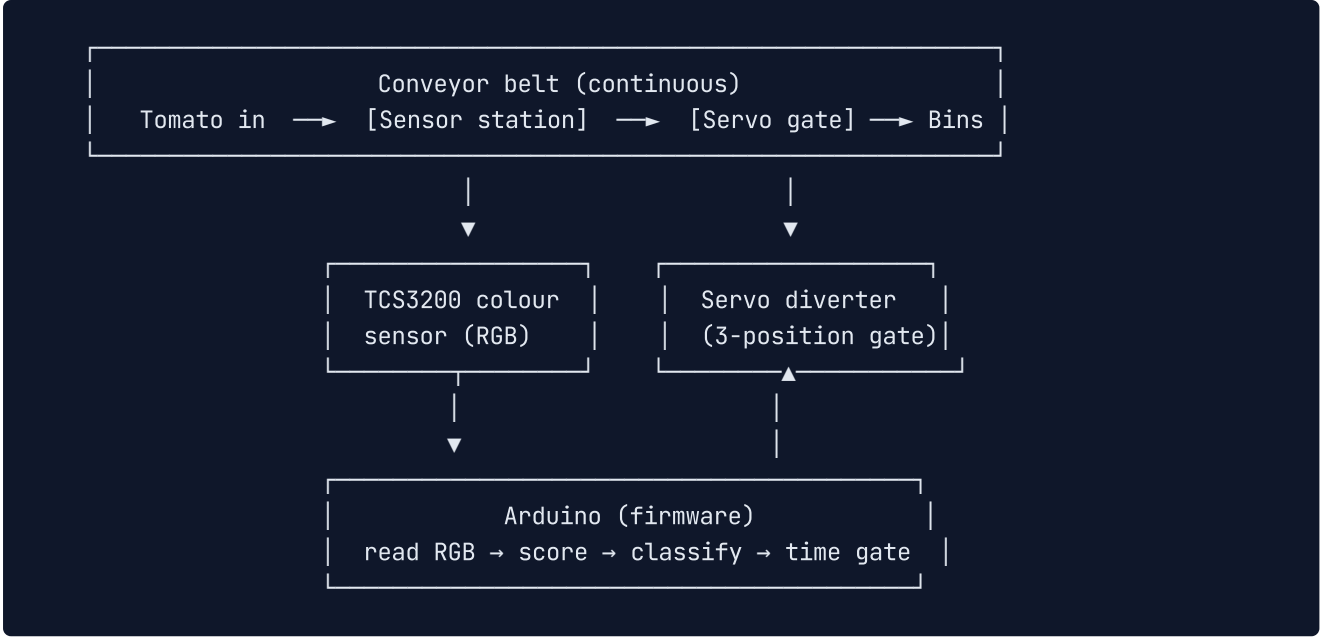
- **RGB colour sensing** — uses a TCS3200 (or compatible) colour sensor to read the red, green, and blue components of each tomato as it passes the sensor station. The firmware averages a small number of readings to reduce noise from a single sample.
- **Three-class ripeness classification** — each reading is reduced to a single ripeness score and classified against two configurable thresholds into *green*, *partially ripe*, or *red*. The thresholds live in a configuration block at the top of the firmware so they can be retuned without touching the classification logic.
- **Servo-driven sorting gate** — a single servo rotates a diverter gate to one of three positions corresponding to the three ripeness bins. The firmware controls the gate timing relative to the conveyor speed so the right tomato lands in the right bin even when the belt is moving.
- **Belt speed awareness** — the firmware uses a configurable belt-speed parameter (and optionally an IR slot sensor) to time the delay between the colour reading and the servo actuation, so the sorting decision is applied to the correct tomato rather than the one immediately after it.

- **Serial diagnostics** — every reading, classification decision, and servo movement is written to the serial monitor in a simple human-readable format, so the system can be tuned and debugged without any external tooling.
- **Modular firmware structure** — the codebase is split into clearly named modules (colour reader, classifier, servo driver, conveyor timing) rather than a single monolithic sketch, so each layer can be tested and replaced independently.

🔌 Hardware Components

Module	Component	Role	Notes
Controller	Arduino UNO or Nano	Main MCU	UNO preferred for classroom builds
Colour sensor	TCS3200 (or TCS230)	RGB colour reading	Frequency output; needs LED illumination
Conveyor	Small DC motor + belt	Moves tomatoes past sensor	Speed controlled via PWM or fixed supply
Actuator	Standard servo (SG90 or MG90)	Drives sorting gate	3-position rotation
Optional	IR slot sensor or photo-interrupter	Tomato position detection	Improves sorting accuracy
Power	5 V / 2 A supply	System power	Servo needs separate supply from Arduino
Frame	Laser-cut or 3D-printed chassis	Holds sensor, gate, bins	Documented in `/mechanical`

🧱 System Architecture



The sensing and actuation happen at two different physical positions on the belt, so the firmware uses a configurable delay (or a position sensor) to apply the sorting decision to the tomato that was actually measured, not the one currently under the gate. This is the part of the project that makes it a real industrial-style control problem rather than a simple "read sensor, move servo" demo.

📂 Repository Structure

```
Arduino-tomato-colour-sorting/
├── README.md                  # This file
├── /firmware
│   ├── tomato_sorter.ino      # Main sketch
│   └── /modules
│       ├── colour_reader.cpp  # TCS3200 driver
│       ├── classifier.cpp     # Ripeness scoring + thresholds
│       ├── servo_driver.cpp   # Gate control
│       └── conveyor_timing.cpp# Belt-speed delay logic
├── /mechanical
│   ├── chassis-overview.md    # Frame design notes
│   └── gate-mechanism.md      # Servo + diverter design
├── /docs
│   ├── calibration-guide.md   # How to baseline the colour sensor
│   └── tuning-guide.md        # Threshold + timing tuning
├── /data
│   └── sample-runs.csv         # Sample readings + classifications
└── LICENSE
```

🚀 Getting Started

Prerequisites

- Arduino IDE 1.8.13 or newer
- Arduino UNO or Nano
- TCS3200 colour sensor, servo, small DC motor or pre-built conveyor
- Separate 5 V supply for the servo (do not power the servo from the Arduino's 5 V pin)

Installation

1. **Clone the repository**
   `bash git clone https://github.com/nicksananton0923-create/Arduino-tomato-colour-sorting.git cd Arduino-tomato-colour-sorting`
2. **Open** `firmware/tomato_sorter.ino` in the Arduino IDE and select the correct board and port.
3. **Install required libraries** (e.g. `Servo.h` is bundled with the IDE; any colour-sensor library is referenced in the firmware).
4. **Wire the components** following the wiring guide in `docs/calibration-guide.md`.
5. **Power the servo from a separate 5 V supply** — sharing the Arduino's 5 V rail can cause brownout resets when the servo moves.
6. **Upload the firmware** and open the Serial Monitor.

🎯 Usage

1. **Calibrate the colour sensor** against real tomato samples of each ripeness class. Use the helper sketch in `firmware/modules/colour_reader.cpp` to print raw RGB values for each sample.
2. **Set the thresholds** in the configuration block at the top of `tomato_sorter.ino`:

```cpp
// === Ripeness classification ===
const int RIPENESS_GREEN_MAX = 300; // score < 300 → green
const int RIPENESS_PARTIAL_MAX = 600; // 300 ≤ score < 600 → partially ripe
// score ≥ 600 → red

// === Conveyor timing ===
const unsigned long SENSOR_TO_GATE_MS = 1200; // delay between reading and actuation
const int SERVO_POS_GREEN = 30;
const int SERVO_POS_PARTIAL = 90;
const int SERVO_POS_RED = 150;
```

3. **Run the conveyor** at a fixed speed. Place a tomato at the input end.
4. The firmware will read the colour, classify, and actuate the servo gate after the configured delay.

5. Watch the Serial Monitor for live readouts of `R, G, B, score, class, servo_pos`.

🎨 Calibration & Tuning

- **Ambient light matters.** The TCS3200 is sensitive to ambient light. Shield the sensor station with a small opaque hood and use the sensor's onboard LED for consistent illumination.
- **Sample several tomatoes per class.** Single-sample calibration is unreliable. Take 5–10 readings per ripeness class and use the average.
- **Servo timing depends on belt speed.** If you change the conveyor motor speed, recalibrate `SENSOR_TO_GATE_MS` empirically by running a known tomato through the system and observing which bin it lands in.
- **Mechanical alignment.** The diverter gate must reach its target position before the tomato arrives. If the gate is still moving when the tomato arrives, the tomato will be deflected incorrectly.

⚠️ Safety & Limitations

- **Mechanical safety.** Moving belts and servo-driven gates can pinch fingers. Use a transparent cover over the gate mechanism during demonstrations.
- **Power supply.** A servo under load can draw bursts of current that crash the Arduino. Always use a separate 5 V supply for the servo and tie the grounds together.
- **Sensor limits.** The colour sensor cannot see through packaging, dirt, or uneven tomato surfaces. Sort washed, unpacked tomatoes for best results.
- **Not for production.** This is an educational prototype. Production-scale sorters use machine vision, multi-sensor fusion, and food-grade materials. This project does not replace them.

🗺 Roadmap

- [ ] Add an OLED display showing the last reading + classification.
- [ ] Add an IR position sensor for tighter sensor-to-gate timing.
- [ ] Add a counter that records how many tomatoes went into each bin per session.
- [ ] Add an ESP32 variant with Wi-Fi dashboard for remote session monitoring.
- [ ] Add a calibration helper sketch that auto-suggests thresholds from a learning set.

📄 License

This project is released under the MIT License unless a separate `LICENSE` file in the repository specifies otherwise.

🍅 Sense → Score → Sort. Reproducible agricultural automation. 🍅 Project by [@nicksananton0923-create](https://github.com/nicksananton0923-create) · Electrical Technology · Renewable Energy · Automation

💧 Industrial Water Consumption Monitoring **Reusable system for recording, validating, analyzing & visualizing industrial water consumption**



📋 Table of Contents

- Overview
- Background
- Key Features
- Repository Structure
- Data Model
- Calculation Methodology
- Validation Rules
- Dashboard Plan
- Getting Started
- Usage
- Data Privacy & Confidentiality
- Roadmap
- License

📝 Overview

The **Industrial Water Consumption Monitoring** project is a reusable, vendor-neutral system for recording water meter readings, calculating consumption between successive readings, validating the data for errors, and visualizing trends and anomalies in a dashboard that can be built in Excel or Power BI. The project is designed for industrial and commercial facilities that need a structured way to track water usage across multiple meters and locations without investing in a commercial metering platform. The output is intended for facility managers, sustainability officers, and operations teams who need to understand where water is being used, where usage is unusual, and where opportunities for reduction exist.

The repository is currently in a structured planning phase: the data model, calculation methodology, validation rules, and dashboard plan are all documented, and the supporting templates are being built incrementally. Every design decision is written down before the corresponding file is created, so readers can see not only *what* the system does but *why* each choice was made. This makes the project a useful reference for anyone learning how to design a small data system the right way —

start with the schema, then the calculation rules, then the validation rules, then the visualization, and only then the dashboard.

The system is intentionally tool-light. Excel and Power BI are evaluated first because they are the most accessible tools for facility teams. Python is recommended only where it adds useful data validation or automation that the spreadsheet cannot do natively. This keeps the barrier to entry low while still leaving room for advanced analysis later.

🔭 Background

Water is one of the most under-managed utilities in industrial facilities. Unlike electricity, which is usually metered at the main incomer and monitored through a utility bill, water is often consumed at many points across a site with only the main meter being read regularly. Sub-meters are frequently installed but rarely read consistently; when they are read, the data is often stored in ad-hoc spreadsheets with no validation, no consistent units, and no clear audit trail. The result is that facility managers usually cannot answer basic questions such as "which area consumed the most water last month?", "is the consumption trend in area X increasing?", or "did the meter in area Y register an anomalous jump that we should investigate?"

This project was built to close that gap in a way that any facility team can adopt without buying new software. It defines a clear data model for water meter records, a calculation methodology that handles cumulative meters correctly (including resets, replacements, and roll-overs), a validation rule set that catches the most common data-entry errors, and a dashboard plan that turns the cleaned data into actionable visualizations. The repository deliberately does not invent consumption readings or claim verified water savings — every number in the project's sample data is clearly labeled as anonymized and synthetic.

✨ Key Features

- **Meter register template** — captures the identity of every meter (meter ID, location, area, equipment, unit, install date, serial number) so that consumption records can be joined to a physical asset. This is the backbone of the system: without a clean meter register, none of the downstream analysis is trustworthy.
- **Consumption log template** — captures each meter reading with date, time, cumulative reading, units, and a free-text notes column. The schema supports both manual readings and automated logger exports.
- **Cumulative-to-consumption calculation** — for cumulative meters, the system computes consumption as the current valid reading minus the previous valid reading for the same meter. The methodology explicitly handles meter resets, replacements, roll-overs, missing readings, and unit changes — all common edge cases that naive spreadsheet calculations get wrong.

- **Validation rule set** — flags missing readings, duplicate readings for the same meter on the same date, negative consumption, implausibly large jumps (anomalies), and inconsistent units. Anomalies are flagged as **potential** issues for investigation, never as confirmed leaks.
- **Daily / weekly / monthly summaries** — the system produces time-bucketed summaries that support comparison across meters, areas, and time periods. These summaries are the input to the dashboard.
- **Trend charts and KPIs** — the dashboard plan defines a small set of useful visualizations (line charts of consumption over time, bar charts of consumption by area, KPI tiles for total consumption, top consumer, biggest change vs. previous period).
- **Anomaly flags** — any reading that breaches a configurable threshold (e.g. consumption more than 3× the rolling average for that meter) is flagged for investigation. The flag is an invitation to investigate, not a confirmed conclusion.
- **Optional target-versus-actual comparisons** — where an authorized target exists for a meter or area, the dashboard can compare actual consumption against the target and surface the variance.

📂 Repository Structure

```
Industrial-water-consumption-monitoring/
├── README.md                       # This file
├── /templates
│   ├── water-meter-register.csv    # Meter master list
│   └── water-consumption-log.csv   # Reading log
├── /data
│   └── sample-water-readings.csv   # Anonymized synthetic dataset
├── /docs
│   ├── data-dictionary.md          # Every column, every unit
│   ├── calculation-methodology.md  # Cumulative meter calculations
│   ├── data-validation.md          # Validation rule set
│   └── dashboard-guide.md          # Excel / Power BI build instructions
├── /dashboard
│   ├── excel-dashboard.xlsx        # (planned)
│   └── powerbi-template.pbit        # (planned)
└── LICENSE
```

🗃 Data Model

The system is built around two core tables:

Table	Granularity	Purpose
`water-meter-register`	One row per meter	Identity + location of each meter

`water-consumption-log`	One row per reading	Each individual meter reading event

The two tables are joined on `meter_id`. The register is the source of truth for what each meter is and where it sits; the log is the source of truth for what each meter has read over time. Any analysis — daily totals, area comparisons, trend charts — starts by joining these two tables.

🧮 Calculation Methodology

For cumulative meters (the most common type), consumption between two successive readings of the same meter is:

```
consumption = current_reading - previous_reading
```

The following edge cases are explicitly handled:

- **Meter reset / replacement:** if `current_reading < previous_reading` and a reset / replacement is documented in the notes column, treat `current_reading` as a new baseline and skip the delta for that interval.
- **Meter roll-over:** if `current_reading < previous_reading` and the meter is documented to have rolled over at its maximum scale, add the maximum scale to the delta.
- **Missing readings:** if a reading is missing for a date, no consumption is computed for that interval; the next valid reading computes the cumulative consumption across the gap.
- **Unit changes:** if a meter's unit changes (e.g. from gallons to liters), the readings before and after the change cannot be directly compared. The system flags this and requires explicit reconciliation before computing consumption across the change point.

Full details are in `docs/calculation-methodology.md`.

✅ Validation Rules

The validation rule set is intentionally simple and explainable:

1. **No duplicate readings** for the same meter on the same date and time.
2. **No negative consumption** unless a documented reset / replacement / roll-over has been recorded.
3. **No missing mandatory fields** — meter ID, date, reading, unit are all required.
4. **No inconsistent units** — the unit column must match the unit declared in the meter register for that meter.
5. **Anomaly flag** — consumption more than 3× the rolling 7-day average for that meter is flagged as a potential issue for investigation, not a confirmed leak.

6. **Date sanity** — reading dates must be within a reasonable window of the recording date (e.g. not in the future, not more than 1 year in the past).

📊 Dashboard Plan

The dashboard plan defines four standard views:

View	Type	Purpose
Total consumption trend	Line chart	Monthly total consumption across all meters
Consumption by area	Stacked bar chart	Monthly consumption broken down by area
Top consumers table	Sorted table	Top 10 meters by total consumption in the period
Anomaly investigation list	Filtered table	All readings flagged as anomalies, ready for review

The dashboard can be built in either Excel (using PivotTables + PivotCharts) or Power BI (using the standard table + chart visualizations). Build instructions are in `docs/dashboard-guide.md`.

🚀 Getting Started

1. **Clone the repository**
   `bash git clone https://github.com/nicksananton0923-create/Industrial-water-consumption-monitoring.git cd Industrial-water-consumption-monitoring`

2. **Read the design documents in order:**
   - `docs/data-dictionary.md` — understand the data model.
   - `docs/calculation-methodology.md` — understand how consumption is computed.
   - `docs/data-validation.md` — understand the validation rules.
   - `docs/dashboard-guide.md` — understand the dashboard plan.

3. **Populate** `templates/water-meter-register.csv` with your facility's meters. Use the data dictionary for the exact column names and units.

4. **Begin recording readings** in `templates/water-consumption-log.csv`. One row per reading.

5. **Build the dashboard** in Excel or Power BI following `docs/dashboard-guide.md`.

🎯 Usage

- **Daily:** technicians record new meter readings in the consumption log.

- **Weekly:** a reviewer runs the validation rules across the consumption log and investigates any flagged anomalies.
- **Monthly:** the dashboard is refreshed and shared with facility management. Consumption trends and anomalies are reviewed in a short monthly meeting.
- **Quarterly / annually:** the meter register is reviewed for accuracy (meters that have been replaced, relocated, or decommissioned are updated).

🔒 Data Privacy & Confidentiality

- **Never commit real facility data to a public repository.** Use the `sample-water-readings.csv` pattern as a guide for anonymizing your own data before sharing any version of this work publicly.
- **Synthetic data only in samples.** All numbers in `data/sample-water-readings.csv` are clearly labeled as synthetic and anonymized.
- **No claims of verified savings.** The system does not claim verified water savings without evidence. Anomalies are flagged as potential issues, not confirmed leaks.

🗺 Roadmap

- [ ] Publish the water meter register + consumption log CSV templates.
- [ ] Publish the data dictionary.
- [ ] Publish the calculation methodology + a worked example.
- [ ] Publish the validation rule set with a Python helper script.
- [ ] Publish the Excel dashboard template.
- [ ] Publish the Power BI template.
- [ ] Translate key documentation into Tamil for bilingual use.

📄 License

This project is released under the MIT License unless the `LICENSE` file in the repository specifies otherwise.

💧 Measure it. Validate it. Visualize it. Then act on it. 💧 Project by [@nicksananton0923-create] (https://github.com/nicksananton0923-create) · Electrical Technology · Renewable Energy · Automation

🚀 Setup Guide — Professional GitHub Profile

This guide walks you through turning the README files in this folder into a polished, professional GitHub presence. Follow it top to bottom; each step is short and produces visible results.

> **Folder contents**
> - `README.md` → goes in the special `nicksananton0923-create/nicksananton0923-create` repo (your profile).
> - `repos/<name>—README.md` → goes inside each matching repository, replacing the existing `README.md`.
> - `PREVIEW.pdf` → for your review only; do not upload.

✅ Pre-flight Checklist

Before you start, make sure you have:

- [ ] A GitHub account logged in at https://github.com (the one named **nicksananton0923-create**).
- [ ] The README files from this folder downloaded to your computer.
- [ ] A simple text editor (VS Code, Notepad, or GitHub's built-in web editor — all work).
- [ ] Your real **LinkedIn profile URL**, **email address**, and (optional) **portfolio URL** ready.

> The README files contain a few placeholders like `[YOUR-LINKEDIN-URL]`, `[YOUR-EMAIL]`, and `[YOUR-PORTFOLIO-URL]`. You will replace these with your real info in **Step 2** before uploading.

📌 Step 1 — Fill In Your Personal Placeholders

Open **every** `.md` file in this folder and replace the placeholders with your real information. Do this *before* uploading so you only do it once.

Placeholders to replace

Placeholder	Replace with	Example

`[YOUR-LINKEDIN-URL]`	Your LinkedIn profile URL	`https://www.linkedin.com/in/nicksananton`
`[YOUR-EMAIL]`	Your contact email	`nicksananton@example.com`
`[YOUR-PORTFOLIO-URL]`	Your portfolio / personal site (or remove the line)	`https://nicksananton.dev`

How to find them

In any text editor (or on GitHub's web editor), use **Find & Replace** (Ctrl+H on Windows / Cmd+H on Mac):

- Find: `[YOUR-LINKEDIN-URL]` → Replace with your LinkedIn URL
- Find: `[YOUR-EMAIL]` → Replace with your email
- Find: `[YOUR-PORTFOLIO-URL]` → Replace with your portfolio URL, or remove that badge line if you don't have one yet

Do this for **every** `.md` **file** in this folder — the profile `README.md` and all 6 files in `repos/`.

> **Tip:** If you don't have a portfolio URL yet, just delete the entire "Portfolio" badge line in the *Connect* sections. Don't leave the placeholder showing on your live profile.

🧑‍💻 Step 2 — Update Your GitHub Profile (Bio + Avatar + Name)

These settings live on your GitHub account, not in the README.

2.1 Set your public name

1. Go to https://github.com/settings/profile
2. Under **Name**, enter your real name (e.g. **Nicksananton** — your brand name).
3. Click **Update profile**.

2.2 Write a short bio

In the same page, under **Bio**, paste:

```
Building Smart Solutions ⚡ | Electrical Technology | Renewable Energy ☀️ | Electronics & Au
```

(Or keep your existing one — it already matches your brand.)

2.3 Set your location

In the same page, under **Location**, enter **Sri Lanka** (or your city if you prefer).

2.4 Upload a professional avatar

Still on the same page, click the avatar circle and upload a clean headshot or your logo. A simple, clear, square image (≥500×500 pixels) works best. Avoid blurry or dark photos — this is the first thing visitors see.

2.5 Add your LinkedIn + email to your GitHub profile

Scroll down on the same settings page; under **Social links**, add:

- LinkedIn → your LinkedIn URL
- (Optional) Personal website → your portfolio URL

Click **Update profile** to save everything.

🌟 Step 3 — Upload the Profile README

This is the most important file — it appears at the top of your GitHub profile page.

3.1 Open the special profile repo

1. Go to https://github.com/nicksananton0923-create/nicksananton0923-create
2. This repository has the same name as your username, which is what makes its `README.md` show on your profile.
3. Click on the existing `README.md` file.
4. Click the **pencil icon** (✏️ Edit this file) in the top-right of the file view.

3.2 Replace the content

1. Open the `README.md` file from this folder on your computer.
2. Copy **all** of its content.
3. Paste it into the GitHub editor, replacing the existing content entirely.
4. Click **Commit changes…** → confirm. Use a clear commit message like:
   `Update profile README with engineering-pro layout`

3.3 Verify

1. Go to https://github.com/nicksananton0923-create
2. You should see your new profile README render at the top of the page — with the wave banner, badges, tech stack table, featured project cards, GitHub stats, and footer banner.

If anything looks broken, click the **pencil icon** on the README again and fix it. Common fixes:
- A badge shows as broken image → check that the URL has no spaces (use `%20` for spaces).

- The wave banner doesn't load → it's served by capsule-render.vercel.app, which is usually reliable. If it fails once, refresh the page.

📦 Step 4 — Upload Each Repo README

Repeat this process for each of the 6 repo README files in the `repos/` folder.

For each repository:

1. Open the repository on GitHub. The 6 target repositories are:
   - `industrial-air-quality-monitor`
   - `Arduino-smart-fire-detection-suppression`
   - `Arduino-tomato-colour-sorting`
   - `Industrial-hvac-maintenance`
   - `Industrial-water-consumption-monitoring`
   - `Industrial-maintenance-request-system`
2. Click on the existing `README.md` file. If there isn't one, click **Add file** → **Create new file** and name it `README.md`.
3. Click the **pencil icon** (✏️ Edit this file).
4. Open the matching file from this folder's `repos/` directory on your computer.
   - The file name pattern is `<repo-name>-README.md` (e.g. `industrial-air-quality-monitor-README.md`).
   - Copy **all** of its content.
5. Paste it into the GitHub editor, replacing the existing content.
6. Click **Commit changes...** with a message like:
   `Update README with professional engineering-pro layout`
7. Refresh the repo page and verify the new README renders correctly.

> **Tip:** If the existing repo already has good content that the new README replaces, you don't lose any work — the old content is still in the git history. You can always go back via **History** if needed.

🏷️ Step 5 — Add Topics (Tags) to Each Repository

Topics help people discover your repositories through GitHub search. Add 4–6 topics to each repo.

How to add topics

1. Open the repository on GitHub.
2. In the right sidebar, find the **Topics** section (under the description).
3. Click the **gear icon** (⚙️) next to Topics.
4. Add the topics below for that repo. Type each one and press Enter.
5. Click **Save changes**.

Suggested topics per repo

Repo	Suggested topics
`industrial-air-quality-monitor`	`arduino`, `air-quality`, `embedded`, `sensors`, `industrial-iot`, `monitoring`
`Arduino-smart-fire-detection-suppression`	`arduino`, `fire-detection`, `safety`, `embedded`, `automation`, `iot`
`Arduino-tomato-colour-sorting`	`arduino`, `automation`, `agriculture`, `color-sensor`, `conveyor`, `embedded`
`Industrial-hvac-maintenance`	`hvac`, `maintenance`, `industrial`, `csv-templates`, `building-services`, `documentation`
`Industrial-water-consumption-monitoring`	`water`, `monitoring`, `data-analysis`, `excel`, `dashboard`, `industrial`
`Industrial-maintenance-request-system`	`cmms`, `javascript`, `maintenance`, `work-orders`, `industrial`, `nodejs`

📝 Step 6 — Update Each Repo Description

The short description under each repo name on your profile page matters a lot — it's the second thing visitors see after the repo name.

How to update

1. Open the repository.
2. In the right sidebar, find **About** (under the description).
3. Click the **pencil icon** (⚙️) next to About.
4. Update the description with one of the suggested texts below (or your own).
5. Tick **Releases**, **Packages**, and **Environments** only if relevant (usually leave unticked).
6. Click **Save changes**.

Suggested descriptions

Repo	Suggested description
`industrial-air-quality-monitor`	Arduino Nano-based industrial air quality monitoring and alert system
`Arduino-smart-fire-detection-suppression`	Arduino-based smart fire detection prototype with conceptual automatic suppression
`Arduino-tomato-colour-sorting`	Arduino-based smart conveyor belt for tomato ripeness detection and automatic sorting
`Industrial-hvac-maintenance`	Professional HVAC, chiller, AHU & cooling system maintenance documentation framework
`Industrial-water-consumption-monitoring`	Industrial water consumption monitoring, trend analysis & reporting dashboard
`Industrial-maintenance-request-system`	Digital maintenance management system for work requests, assignments, and work order tracking

---

📌 Step 7 — Pin Repositories to Your Profile

Pinned repos show at the top of your profile, right under the README. Pin 6 of your 7 repos (the 7th is the profile README repo itself, which doesn't need pinning).

How to pin

1. Go to https://github.com/nicksananton0923-create
2. Scroll to the **Pinned** section (just below the README).
3. Click **Customize your pins**.
4. Select these 6 repositories (in this order for visual balance):
   - `industrial-air-quality-monitor` ← only one with a star
   - `Industrial-hvac-maintenance`
   - `Industrial-maintenance-request-system`
   - `Arduino-smart-fire-detection-suppression`
   - `Arduino-tomato-colour-sorting`
   - `Industrial-water-consumption-monitoring`
5. Click **Save pins**.

> The pinning order matters because GitHub shows them left-to-right, top-to-bottom. Putting the starred repo first makes the profile look more established at first glance.

🔗 Step 8 — (Optional) Add a Profile README Banner Image

If you want even more polish, you can add a custom banner image instead of the auto-generated one.

1. Create a banner image (1280×320 px) with your name + tagline. Tools that work well:
   - https://canva.com (search "GitHub banner")
   - https://bannerator.vercel.app (auto-generates from text)
2. Save it as `banner.png`.
3. Upload it to your `nicksananton0923-create/nicksananton0923-create` repo (drag-and-drop on the GitHub file browser).
4. Edit the profile `README.md` and replace the first `<img>` line (the capsule-render banner) with: `markdown <img src="banner.png" width="100%" alt="header banner" />`
5. Commit the change.

✅ Step 9 — Final Verification

Open https://github.com/nicksananton0923-create in a **fresh / incognito window** (so you see what visitors see, not your logged-in view) and check:

- [ ] Profile README renders fully — banner, name, badges, tech stack, featured projects, stats, footer banner.
- [ ] No `[PLACEHOLDER]` text is visible anywhere on the profile or in any repo README.
- [ ] All 6 pinned repositories appear under the README with their new descriptions.
- [ ] Each pinned repo shows a small language tag and stars/forks count.
- [ ] All badge images load (no broken image icons).
- [ ] GitHub stats card shows on the profile (sometimes takes a few minutes on first load — refresh once).

If anything is broken, edit the corresponding README and fix it. Commit messages like `Fix LinkedIn badge URL` are perfectly fine — small fixes are how a profile improves over time.

🛠 Maintenance Tips

- **Commit regularly.** When you update a project, push the change to GitHub the same day. A profile with recent commits looks alive; one with no commits for 6 months looks abandoned.

- **Keep the README in sync with the project.** If you add a new file to a repo, mention it in the README. A README that lists files that don't exist (or omits files that do) is worse than no README.
- **Star repositories you learn from.** Your starred list shows what you're studying — it's a passive signal of your interests. Employers look at this.
- **Update the *Currently Learning* badges every few months.** As you finish learning PLCs or AutoCAD, replace those badges with the next thing. A current "currently learning" section shows you're actively growing.
- **Add new repos over time.** Each new project gets the same README pattern as the existing ones. Consistency across the portfolio is what makes it look professional.

❓ Troubleshooting

Problem	Fix
README shows raw markdown, not rendered	Make sure the file is named exactly `README.md` (capital letters, .md extension).
Badge shows broken image	Check the URL — copy-paste it into a browser; if it 404s, find a replacement at https://shields.io.
Wave banner doesn't load	capsule-render.vercel.app may be temporarily down. Refresh, or replace with a local image (see Step 8).
GitHub stats card shows zeros	The stats card needs your account to have some activity. If you have very few commits, it'll just show zeros — that's fine, it will fill in over time.
Pinned repo not showing	Make sure the repo is **public**. Private repos don't show on a public profile.
Old README content still showing	Hard refresh the page (Ctrl+Shift+R) — GitHub sometimes caches.

🎉 You're Done

Once you finish all the steps, your GitHub profile will show:

- A clean **wave banner** with your name and tagline.
- A row of identity badges (GitHub, Location, Focus, Status) and visitor counter.
- A short, well-written **About Me** that positions you as an electrical technology student with a documentation-first mindset.
- A clean **Tech Stack** table split into Electrical / Industrial / Embedded / Data columns.
- Six **featured project cards** in a 2×3 grid, each with badges and a link.

- A **Practical Experience** section showing your real internships.
- A **Currently Learning** section with 2026 goals.
- Live **GitHub stats + streak + top languages** cards.
- A clean **Connect** section with LinkedIn / Email / Portfolio badges.
- Six pinned repos with professional descriptions and topic tags.

That's a senior-engineer-grade GitHub profile — built in an afternoon.

⚡ Now go make your first commit of the new profile. ⚡